

Notes Receiver changes 082126

```
#define CRC32_INIT          ((uint32_t)-1l)
#define DATA_TO_CHECK_LEN      9
#define CRC32_LEN            4
#define TOTAL_LEN             (DATA_TO_CHECK_LEN + CRC32_LEN)
static uint8_t src[TOTAL_LEN] = { 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39, 0x00,
0x00, 0x00, 0x00 };

static uint8_t src1[TOTAL_LEN] = { 0x39, 0x38, 0x37, 0x36, 0x35, 0x34, 0x33, 0x32, 0x31, 0x00,
0x00, 0x00, 0x00 };
static uint8_t srcR[TOTAL_LEN];
// This uses a standard polynomial with the alternate 'reversed' shift direction.
// It is possible to use a non-reversed algorithm here but the DMA sniff set-up
// below would need to be modified to remain consistent and allow the check to pass.

static uint32_t soft_crc32_block(uint32_t crc, uint8_t *bytp, uint32_t length) {
    while(length--) {
        uint32_t byte32 = (uint32_t)*bytp++;

        for (uint8_t bit = 8; bit; bit--, byte32 >>= 1) {
            crc = (crc >> 1) ^ (((crc ^ byte32) & 1ul) ? 0xEDB88320ul : 0ul);
        }
    }
    return crc;
}

while (true) {
    if (invert == false)
    {
        // Invert the input logic hardware-wide (if using as an input)
        gpio_set_inover(PIO_RX_PIN, GPIO_OVERRIDE_INVERT);

        invert = true;
    }

    while (c = uart_rx_program_getc(pio, sm) != 0x2c)
    {
    }

    for(i=0;i < TOTAL_LEN;i++)
    {
        c = uart_rx_program_getc(pio, sm);

        putchar(c);

        srcR[i] = c;
    }

    crc_res = soft_crc32_block(CRC32_INIT, srcR, DATA_TO_CHECK_LEN);
```

```
printf("0x%x\n",crc_res);

//9-10xfea0fdfe    1-9    0x340bc6d9
//                fefda0fe        d9c60b34

}
```

```
Welcome to minicom 2.8

OPTIONS: I18n
Port /dev/ttyUSB0, 06:20:18

Press CTRL-A Z for help on special keys

Starting PIO UART RX example
```

crc32 was generated correctly on incoming data 123456789

```
Welcome to minicom 2.8

OPTIONS: I18n
Port /dev/ttyUSB0, 05:45:52

Press CTRL-A Z for help on special keys

Starting PIO40x340bc6d9xample
123456789UÆ
```

crc32 was generated correctly on incoming data 123456789

```
0xf0b1f44
%Â
B0x4bd373d5
À 0xb3dfb2fd
030o(0xe514f702
i@, 0xe9262c6e
0x4d540x340bc6d9
123456789UÆ
```